Qualifying the HTTP DSL

What each import actually gives you, where names collide, and the worst case

Okapi

July 16, 2026

Introduction

Okapi's HTTP codec DSL is spread across roughly twenty modules — one core engine (`Okapi.Tree`), shared `Headers` and `Body` types, per-side `Request`/`Response` facades, and a full RFC 9651 Structured Field Values implementation nested underneath. This document maps exactly which import gets you what, which names collide, why, and what resolves it.

The rule the whole design follows now: **design around the assumption that names will be qualified.** `Okapi` is the generic mode/framework layer only — `Contract`, `Endpoint`, `Handle`, `Client`, `Link`, `Morph`, `Cases`, and the generic `Tree` codec engine. None of it is HTTP-DSL-specific. **Everything HTTP-DSL-shaped is reached through exactly one of three qualifiers**: `Okapi.HTTP` (`HTTP.`) for what's genuinely shared between sides (`field`, `json`, `contentType`, …), `Okapi.HTTP.Request` (`Req.`) for what's genuinely Request-only (the type itself, `req`/`method`/`path`/ `query`, `seg`/`param`, `cookie`), and `Okapi.HTTP.Response` (`Res.`) for what's genuinely Response-only (`res`/`status`, `setCookie`, `Attributes`). A module reaches for whichever combination it needs — one, two, or all three — and qualifies every one of them. This isn't fighting for an unqualified shortcut and then documenting the exceptions; it's the design itself, and it makes every genuinely different qualifier level mean something specific rather than being an artifact of what happened to fit under `Okapi` without colliding.

Every claim here was checked against the real library, not asserted from memory — the worked example at the end is real, compiling code, verified against `lib/src` with GHCi (`cabal repl lib:okapi`, `:load`), not simplified past the point of actually type-checking.

Generic-deriving (`Path.derived`, `Query.derived`, `Okapi.HTTP.Headers.derived`, `GPath`/`GQuery`/`GHeaders`, `LitF`, `ConstF`) is out of scope for this pass — it has its own qualification story and will get its own document.

Part 1 — What bare `import Okapi` gives you

A single unqualified `import Okapi` brings in the mode/framework layer and the generic codec engine — nothing HTTP-DSL-shaped at all:

- **Contracts and responses**: `Contract(..)`, `(:->)`, `(:-<)`, `Cases`, `cases`, `getResponses`, `parseResponses`, `printResponses`.
- **The whole Endpoint/Handle/Client/Link/Morph layer**: `Endpoint(..)`, `endpoint`, `scope`, `route`, `catchAll`, `Handle(..)`, `handle`, `mount`, `run`, `endpoints`, `Transformer(..)`, `client`, `clientVia`, `fetch`, `clientFor`, `Link(..)`, `links`, `linksVia`, `morph`, `openApi`, `contractToOpenApi`.
- **The generic `Tree` engine**: `SymTree`, `Leaf(..)`, `Info(..)`, `HasLeaf(..)` (which brings the `leaf` method itself into scope), `(=.)`, and the leaf combinators `int`, `int16`, `int32`, `int64`, `integer`, `bool`, `float`, `double`, `scientific`, `text`, `day`, `localTime`, `utcTime`, `timeOfDay`, `uuid`.

Notably absent, entirely: `Request`, `Response`, `req`/`res` and their families, method/status singletons, `seg`/`param`/`field`/`json`/`cookie`/ `setCookie`, `IsoJson`, and everything else HTTP-DSL-shaped. None of it

lives here — see Parts 2 and 3.

That's the whole surface. `Okapi` alone can define the mode/contract machinery, but it can't build a single request or response codec.

Part 2 — `Okapi.HTTP`: the genuinely shared operations

`field`, `field'`, `field_`, `contentType`, `fieldStruct`, `fieldBareItem`, `fieldItem`, `fieldList`, `fieldDict` (headers), `json`, `jsonValue`, `noContent` (body), `MediaType(..)`, `None(..)`, and `IsoJson` are genuinely **not** Request-or-Response-specific — `Okapi.HTTP.Headers`/ `Okapi.HTTP.Body` tag `Headers`/`Body` with a phantom `ForRequest`/ `ForResponse` marker (`Okapi.HTTP.Side`), and these combinators are free in that phantom: the exact same function works for either side, resolved by ordinary type inference from context, the same mechanism that resolves `mempty` or `Nothing`. They live in `Okapi.HTTP`, imported `qualified as HTTP`, and *only* there — not duplicated into `Req.`/`Res.` any more, so there's exactly one name for each of these, not two paths to the same binding.

```
import Okapi.HTTP qualified as HTTP

reviewReqHeaders = do
    HTTP.field_ "x-service" "reviews"
    HTTP.contentType HTTP.JSON
    ...

reviewOkHeaders = do
    HTTP.contentType HTTP.JSON
    limit <- rp2 =. HTTP.field "x-ratelimit-limit" int
    ...
```

`cookie`/`cookie'` (request-only), `setCookie` (response-only), and `form` (request-only body) are different in kind — genuinely side-pinned at their GADT constructor — so they live on `Req.`/`Res.` respectively, not `HTTP.` (Part 3).

An `Okapi/HTTP.hs` facade like this one was actually built and removed earlier in this design process, before `field`/`json`/etc. were unified into one shared definition each — at the time it had no real consumer. Reviving it here isn't reversing that call; the circumstances are different now that there's a concrete, well-defined set of genuinely shared operations that need exactly this kind of home.

Part 3 — `Okapi.HTTP.Request`/`Okapi.HTTP.Response`: the side-pinned operations

Each facade holds only what's genuinely specific to its side — nothing shared lives here any more (Part 2), and there's no more curated subset of anything: qualification already disambiguates, so there's no reason to limit what's exposed.

`Okapi.HTTP.Request` (`qualified as Req`): `Request(..)`, `req`, `reqGET`, `reqPOST`, `reqPUT`, `reqDELETE`, `reqPATCH`, `reqHEAD`, `reqOPTIONS`, `reqCONNECT`, `reqTRACE`, the narrowing functions `method`/`path`/`query`/`headers`/`body`, **all 9** method singletons (`KnownMethod(..)`, `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `HEAD`, `OPTIONS`, `CONNECT`, `TRACE`), `seg`/`seg_`/`lit`/`segs`, `param`/ `param'`/`param_`/`flag`/`flag'`/`list`/`list'`/`ArrayStyle(..)`, and the two side-pinned header/body combinators `cookie`/`cookie'` and `form`.

`Okapi.HTTP.Response` (`qualified as Res`): `Response(..)`, `res`, **all** `res100`…`res511` pre-built response codecs, `headers`/`body` narrowing, **all** ~**47** status singletons (`KnownStatus(..)`, `S100`…`S511`, `SomeKnownStatus(..)`, `allKnownStatuses`), the side-pinned `setCookie`, and the curated `attr`/`attr'`/`secure`/`httpOnly` (4 of the 9 `Attributes` combinators — `Attributes` is only ever used building `Set-Cookie`, so it rides along

with `Response`, not `Request`; the `Attributes` curation itself is unrelated to the method/status change and unaffected by it — see 3.6).

Note the real tradeoff versus the previous design: a module that only builds *one* side now needs **two** imports, not one — its side's facade plus `Okapi.HTTP` for the shared field/body combinators. That's a genuine cost of committing to "shared things get their own honest qualifier," paid even by the simplest single-side module. It buys back a name that always means the same thing regardless of which qualifier reached it.

3.1 `headers`/`body` record-update is ambiguous with too few fields present

`Request` and `Response` share field names (`headers`, `body`), which is normally invisible — `DuplicateRecordFields`-style disambiguation resolves it as long as the update also touches a field unique to one side:

```
-- fine: `path`/`query` only exist on Request, so GHC infers the rest
Req.req { path = reviewPath, query = reviewQuery, headers = ..., body = ... }
```

But an update touching *only* shared field names has nothing to disambiguate with:

```
-- ambiguous: `headers` and `body` both exist on Request and Response
Res.res200 { headers = reviewOkHeaders, body = HTTP.json @Review }
```

Two fixes, both verified against the compiler:

```
-- 1. Narrowing functions instead of record update -- each has a concrete,
--    already-resolved type, so there's nothing to disambiguate.
reviewOk = Res.body (HTTP.json @Review) (Res.headers reviewOkHeaders Res.res200)

-- 2. Qualify the *field name* itself, keep record-update syntax.
reviewOk = Res.res200 { Res.headers = reviewOkHeaders, Res.body = HTTP.json @Review }
```

3.2 Qualifying the field name resolves ambiguity directly — no rewrite needed

This is worth calling out on its own: `Res.headers` inside `{ }` isn't a name search at all — it's already a fully resolved reference to *one specific field label*, the one `Okapi.HTTP.Response` exports under that name. GHC's usual "ambiguous record update" resolution tries to find a *unique* parent type whose field set contains every *unqualified* name written in the braces; a qualified field name skips that search entirely, since there's nothing left to guess. Verified directly:

```
check1 = res200 { Res.headers = myReqHeaders, Res.body = json @Text }
-- :: Response (KnownStatus 200) Int (IO Text)  -- compiles clean
```

Since a module needing both sides now always imports both facades only qualified (there's no unqualified affordance to fall back on any more — Part 3's whole point), the bare field names aren't in scope *at all*, not "ambiguous" — so `Req.req { path = ... }` doesn't even get as far as an ambiguity error; `path` resolves to whatever bare `path` happens to still be in scope from somewhere else entirely (`URI`'s, if `Okapi.Mode.Link` is in scope), giving a confusing type-mismatch error instead of "not in scope." The practical rule: qualify the field name unconditionally, `Req.req { Req.path = ... }`, every time — not just when GHC complains.

3.3 `path` collides with `URI`'s `path` field — in record-update syntax only

`Okapi.Mode.Link`'s `URI { path :: Text, query :: Text }` uses `NoFieldSelectors`, same as `Request`/`Response` — so it generates no top-level `path` function (bare `path` unqualified is *not* ambiguous as a plain function call, since `URI` contributes no such binding). But record-update syntax doesn't work that way — same mechanism as 3.1/3.2:

```
-- silently resolves to URI's path and fails to type-check, since only
-- Req is imported qualified: `path` is a field of both Request and URI
Req.reqDELETE { path = deleteReviewPath }
```

```
-- fine: qualify the field
Req.reqDELETE { Req.path = deleteReviewPath }

-- also fine: the narrowing function has a concrete type already
Req.path deleteReviewPath Req.reqDELETE
```

3.4 `param`/`flag`/`flag'`/`list` mean different things at different levels

Query-string parameters (`Okapi.HTTP.Request.Query`) and RFC 9651 item parameters (`Okapi.HTTP.Structured.Parameter`) are conceptually unrelated — a `?limit=5` query parameter and a `;window=60` Structured Field parameter — but they share combinator names almost exactly: `param`, `param'`, `param_`, `flag`, `flag'`. `Okapi.HTTP.Headers.Attributes` (Set-Cookie attributes) piles a *third* `flag`/`flag'` pair on top of that:

```
import Okapi.HTTP.Structured.Parameters qualified as Params
import Okapi.HTTP.Headers.Attributes qualified as Attr
```

`Req.param`/`Req.flag'` reach the query-string versions; Structured parameters go through `Params.param`/`Params.flag'`; Set-Cookie flags go through `Attr.flag`/`Attr.flag'` (or `Res.attr`/`Res.secure`/ `Res.httpOnly` for the 4 curated ones, 3.6). In one function that touches all three — exactly the shape of the worked example below — every one of these needs its own qualifier to stay unambiguous.

3.5 `item`/`list`/`dict` exist at two levels of Structured Field Values

`Okapi.HTTP.Structured` exports `item`, `list`, `dict` as the three ways to wrap an `Item`/`List`/`Dictionary` codec into a full `Structured` value. `Okapi.HTTP.Structured.List` *also* exports `item` (a single List member, different type entirely). Both are genuinely useful in the same header definition — `fieldStruct`/`fieldList`/`fieldDict` already do the `Structured`-wrapping for you, so most code never touches `Struct.item`/ `Struct.list`/`Struct.dict` directly, but the inner `Item`/`List`/`Dictionary`/`Parameters` modules are unavoidable the moment a header value has any internal structure at all:

```
import Okapi.HTTP.Structured.Item qualified as Item
import Okapi.HTTP.Structured.List qualified as SList
import Okapi.HTTP.Structured.Dictionary qualified as Dict
```

3.6 `Body` is unified with `Headers` — same phantom, same shape, plus two new pieces

`Okapi.HTTP.Request.Body` and `Okapi.HTTP.Response.Body` used to each define their own separate `Body` GADT with duplicated `Raw`/`Json`/ `NoContent` logic. They're now both `type` aliases — `RequestBody = Body ForRequest`, `ResponseBody = Body ForResponse` — over one shared `Okapi.HTTP.Body` core tagged with the same `Okapi.HTTP.Side` phantom `Headers` uses; `Form` is pinned to `Body ForRequest` right at its constructor, the same way `Cookie`/`SetCookie` are pinned on the `Headers` side. Two things came out of that unification worth calling out:

- `noContent`'s value type is a dedicated `None` (`data None = None`) instead of `()` — a no-body response reads as its own explicit thing. `None` genuinely applies to both sides (it's free in the phantom, unlike `Form`) — used for a request body in `getReview` and a response body in `deletedRes` in the worked example below.
- A new `jsonValue` combinator, decoding/encoding a body as a structured `Aeson.Value` directly — no `FromJSON`/`ToJSON`/`ToSchema` instances needed, for callers who want dynamic JSON without a domain type.

`raw` for bodies is the one thing that *didn't* get folded into `Okapi.HTTP` — `Headers` already has its own `raw`, and pulling both into the same shared surface would recreate a collision. It stays reachable only through

`Okapi.HTTP.Request.Body`/`Okapi.HTTP.Response.Body` (or the core `Okapi.HTTP.Body`/`Okapi.HTTP.Headers` directly).

Set-Cookie `Attributes` (`attr`/`attr'`/`secure`/`httpOnly`, curated 4 of 9) is a separate, smaller curation that's *not* affected by dropping the method/status curation in Part 3 — it's a judgment call about a small, rarely-extended set, not a mechanical "expose everything" case.

3.7 `Data`, `Failure`, `Result`, and `Wai.Response` for anything past the DSL

The moment code steps past *defining* a contract into *consuming* one — rendering a `Cases` value with `printResponses`, parsing an incoming one with `parseResponses`, or just naming the decoded-value type — the record modules that hold decoded/error/raw-result shapes come into play, and none of them are reachable via `Okapi` or any of the three HTTP qualifiers:

```
import Okapi.Record.Data qualified as Data
import Network.Wai qualified as Wai
```

3.8 `raw`, `parser`, `printer`, `parseExact` are everywhere

Nearly every leaf module — `Path`, `Query`, the core `Headers`, `Structured`, `Item`, `List`, `Dictionary`, `Parameters`, `Attributes`, the core `Body` — exports its own `raw`, and most export `parser`/`printer` (`Okapi.Tree` itself also exports generic `parser`/`printer`). None of these are re-exported by `Okapi` or any of `HTTP`/`Req`/`Res` (3.6). This isn't usually a *conflict* to resolve so much as a standing fact: reaching for the escape-hatch pass-through (`raw`) or the low-level codec functions on any leaf module always means a qualified import. See the reference table below for the full list.

Part 4 — The naming convention this all sits on

Two rules, applied consistently:

1. **Types and constructors stay fully spelled out.** `Request`, `Response`, `Headers`, `Structured`, `Attributes`, `Dictionary`, `Parameters`, `KnownMethod`, `KnownStatus` — never abbreviated.
2. **Term-level smart constructors abbreviate when the full word is long enough to matter**, using a commonly recognized short form: `request` → `req`, `response` → `res`, `segment`/`segment_`/`segments` → `seg`/`seg_`/`segs`, `attribute`/`attribute'` → `attr`/`attr'`, `dictionary` → `dict` (and `fieldDictionary` → `fieldDict` to match). Short names stay as they are (`field`, `param`, `flag`, `list`, `item`, `member`, `status`, `method`, `raw`) — there's nothing to gain by shortening a four-letter word.

One combinator, `bareItemEq`, broke rule 2 during an earlier pass — every other "assert a fixed, known value" combinator in the DSL (`field_`, `param_` twice over, `seg_`) uses an underscore suffix on its "decode freely" sibling; `bareItemEq` sat right next to `bareItem` with a different naming style for the identical shape. It's now `bareItem_`.

The `Body` unification (3.6) added two more names under the same rules: `None` is a type, so it stays fully spelled rather than becoming something cryptic; `jsonValue` is a term-level combinator short enough already that there's nothing to abbreviate, matching `json`/`noContent`/`form` right next to it. `Okapi.HTTP.Side`'s `ForRequest`/`ForResponse` are phantom marker types, so — same rule as `Request`/`Response` — fully spelled.

Now that qualification is the assumed default rather than something to minimize, a follow-up naming pass (renaming `reqGET`/`res200`-style names to something shorter, REST-idiomatic, and leaning on the qualifier for disambiguation — `Req.get`, `Res.ok`) is a real, separate possibility, not attempted in this pass. It has its own puzzle to solve first: some short names are already spoken for elsewhere in the same qualified namespace (`noContent` is already the body combinator, so a REST-style `Res.noContent` for 204 would collide with it).

Part 5 — Reference: the rest

Functions that exist and are real, but don't naturally show up in a typical contract — the low-level codec plumbing and a couple of specialized Structured Field Value leaf types. Method and status singletons no longer have a "curated vs the rest" split (Part 3) — everything `Okapi.HTTP.Request.Method`/`Okapi.HTTP.Response.Status` define is reachable straight from `Req.`/`Res.`.

Name	Module	Signature (abbreviated)	Purpose
`raw`	`Path`, `Query`, `Headers`, `Structured.Item`, `.List`, `.Dictionary`, `.Parameters`, `Headers.Attributes`, `Okapi.HTTP.Body` (and its `Request.Body`/`Response.Body` aliases)	`Tree t ctx ctx`	Pass the whole context through unconstrained, per module.
`parser` / `printer`	same modules, plus `Okapi.Tree` itself	`Tree t i o -> Parser/Printer t _`	The low-level codec functions every `field`/`param`/etc. combinator is built from.
`parseExact`	`Path`, `Query`, `Headers`, `Structured`, `.Item`, `.List`, `.Dictionary`	`Tree t i o -> ctx -> Either (Either err leftover) o`	Require full consumption instead of tolerating leftover.
`Method.method`, `Method.raw`, `Method.parse`, `Method.print`	`Okapi.HTTP.Request.Method`		Build/inspect a `Method` codec directly (`method`/`print` shadow `Prelude.print` if imported unqualified — the source hides `Prelude.print` itself for this reason).
`Status.status`, `Status.raw`, `Status.parse`, `Status.print`	`Okapi.HTTP.Response.Status`		Build/inspect a `Status` codec directly.
`Attr.maxAge`, `Attr.domain`, `Attr.path`, `Attr.flag`, `Attr.flag'`	`Okapi.HTTP.Headers.Attributes`		The 5 `Attributes` combinators beyond the 4 curated onto `Res.` (`attr`/`attr'`/`secure`/`httpOnly`) — still its own small, deliberate curation (3.6), unrelated to the method/status change.

Name	Module	Signature (abbreviated)	Purpose
`Struct.item`, `Struct.list`, `Struct.dict` used standalone	`Okapi.HTTP.Structured`	`Tree Item/List/Dictionary a a -> Tree Structured a a`	The general escape hatch `fieldStruct` is sugar over — reach for this directly when a header value needs to switch between Item/List/Dictionary shapes generically.
`BareItem.displayString` / `DisplayString`	`Okapi.HTTP.Structured.BareItem`	`DisplayString`	RFC 9651 §3.3.8 Display String — percent-encoded, non-ASCII-safe text, distinct from the plain quoted `Text` (`sf-string`).
`BareItem.byteSequence` / `ByteSequence`	`Okapi.HTTP.Structured.BareItem`	`ByteSequence`	RFC 9651 §3.3.5 — base64, colon-delimited arbitrary bytes.
`BareItem.hasNonCanonicalInteger`	`Okapi.HTTP.Structured.BareItem`	`ByteString -> Bool`	Flags RFC-legal-but-non-canonical integer syntax (leading zeros, `-0`) — used internally to exclude those shapes from round-trip properties.
`BareItem.renderInner` / `parseInnerToList`	`Okapi.HTTP.Structured.BareItem`	—	The primitives `List`'s inner-list handling is built from; only needed directly for hand-rolled inner-list logic outside `List`'s own combinators.
`SList.innerParser` / `innerPrinter`	`Okapi.HTTP.Structured.List`	`Tree InnerItems i o -> Parser/Printer InnerItems _`	Test/inspect an `InnerItems` value (the space-separated contents of one inner list) directly, the way `parser`/`printer` do for `Item`/`List` themselves.
`knownMethodToStd` / `extractMethod` / `knownStatusToHTTP` / `extractStatus`	`Method`, `Status`	—	Convert to/extract from the underlying `http-types` representations.

Part 6 — The worked example

A three-endpoint "product reviews" API, written to use as much of the DSL as naturally fits in one place while staying inside the scope from Parts 1–5 (generic-deriving excluded, per the note above). It compiles as-is against this library — verified with `cabal repl lib:okapi` and `:load`.

```haskell
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeApplications #-}

module MaxQual where

import Data.Aeson qualified as Aeson
import Data.Function ((&))
import Data.OpenApi (ToSchema)
import Data.Text (Text)
import Data.UUID (UUID)
import Data.ByteString (ByteString)
import Network.HTTP.Types qualified as Types
import Data.Maybe (fromMaybe)
import GHC.Generics (Generic)

-- The generic mode/framework layer, plus the fully generic Tree engine --
-- Contract/:->/:-</Cases/cases/getResponses/parseResponses/printResponses,
-- and int/text/uuid/bool/integer/(=.)/Leaf. None of this is
-- Request-or-Response-specific, so it's always safe to leave bare.
import Okapi

-- The genuinely shared HTTP DSL -- field/contentType/json/etc. are one
-- polymorphic definition each, not really either side's, so they get
-- their own honest qualifier instead of borrowing Req./Res. for
-- something that isn't really either side's job.
import Okapi.HTTP qualified as HTTP

-- Everything genuinely Request-specific: the Request type itself,
-- req/reqGET/reqDELETE, method/path/query/headers/body narrowing,
-- seg/param, and the two side-pinned combinators HTTP doesn't carry
-- (cookie/cookie', request-only).
import Okapi.HTTP.Request qualified as Req

-- Same for the Response side (setCookie/attr family, response-only).
import Okapi.HTTP.Response qualified as Res

-- Method.method itself (the smart constructor, not a singleton) only
-- lives in the defining module.
import Okapi.HTTP.Request.Method qualified as Method

-- Query: needed only for the ArrayStyle constructors not re-exported
-- (Exploded/CommaDelimited/SpaceDelimited/PipeDelimited).
import Okapi.HTTP.Request.Query qualified as Query

-- Set-Cookie attributes beyond the curated 4 (attr/attr'/secure/httpOnly,
-- reached via Res.).
import Okapi.HTTP.Headers.Attributes qualified as Attr
```

```haskell
-- Structured Field Values (RFC 9651) -- none of this is Request-or-
-- Response-specific at all, so none of it needs Req./Res./HTTP.
-- qualification.
import Okapi.HTTP.Structured qualified as Struct
import Okapi.HTTP.Structured.BareItem qualified as BareItem
import Okapi.HTTP.Structured.Item qualified as Item
import Okapi.HTTP.Structured.List qualified as SList
import Okapi.HTTP.Structured.Dictionary qualified as Dict
import Okapi.HTTP.Structured.Parameters qualified as Params

-- For the parseResponses/printResponses demo -- neither of these record
-- modules is reachable via Okapi at all.
import Network.Wai qualified as Wai
import Okapi.Record.Data qualified as Data

-- ------------------------------------------------------------------------------
-- Domain type
-- ------------------------------------------------------------------------------

data Review = Review
    { reviewId :: Int
    , stars    :: Int
    , comment  :: Text
    }
    deriving (Generic, Show, Aeson.FromJSON, Aeson.ToJSON, ToSchema)

-- ------------------------------------------------------------------------------
-- Endpoint 1: GET /v2/products/{productId}/reviews/{reviewId}
-- ------------------------------------------------------------------------------

clientToken :: BareItem.Token
clientToken = fromMaybe (error "bad literal token") (BareItem.mkToken "okapi-v2")

-- `&` chains every narrowing function (`Req.path`/`Req.query`/`Req.headers`/
-- `Req.body`) left to right, and `BlockArguments` lets each one take its
-- `do` block directly -- no separate named binding, no parens, no `where`
-- (converted to a leading `let` since `where` can't attach to a bare
-- argument expression).
getReview = Req.req
    & Req.path do
        Req.seg_ int 2
        Req.lit "products"
        pid <- fst =. Req.seg "productId" uuid
        Req.lit "reviews"
        rid <- snd =. Req.seg "reviewId" int
        pure (pid, rid)
    & Req.query do
        let qp1 (a,_,_,_) = a
            qp2 (_,b,_,_) = b
            qp3 (_,_,c,_) = c
            qp4 (_,_,_,d) = d
        verbose <- qp1 =. Req.flag' "verbose"
        limit <- qp2 =. Req.param' "limit" int
```

```
        tagFilter <- qp3 =. Req.list' Query.Exploded "tag" text
        Req.param_ "api" int 2
        fixedFmt <- qp4 =. Req.param "format" text
        pure (verbose, limit, tagFilter, fixedFmt)
    & Req.headers do
        let hp1 (a,_,_,_,_,_,_) = a
            hp2 (_,b,_,_,_,_,_) = b
            hp3 (_,_,c,_,_,_,_) = c
            hp4 (_,_,_,d,_,_,_) = d
            hp5 (_,_,_,_,e,_,_) = e
            hp6 (_,_,_,_,_,f,_) = f
            hp7 (_,_,_,_,_,_,g) = g
        -- Every field-family combinator here is `HTTP.` -- shared, not
        -- either side's -- only `cookie`/`cookie'` (request-only) stay
        -- `Req.`.
        HTTP.field_ "x-service" "reviews"
        HTTP.contentType HTTP.JSON
        apiKey <- hp1 =. HTTP.fieldBareItem "x-api-key" (BareItem.token :: Leaf BareItem.BareItem BareI
        -- The fixed marker token plus a real build-number parameter
        -- riding along with it -- same shape as the library's own
        -- `taggedParam` example: 'Item.bareItem_' composed with a
        -- value-producing sibling keeps the whole thing's input free, no
        -- discard-projection needed. Inlined via BlockArguments too.
        buildNum <- hp2 =. HTTP.fieldItem "x-client" do
            Item.bareItem_ (BareItem.unToken clientToken)
            n <- Item.params (Params.param "build" integer)
            pure n
        traceId <- hp3 =. HTTP.field' "x-trace-id" uuid
        prefs <- hp4 =. HTTP.fieldDict "prefer"
            ( (,) <$> (fst =. Dict.member "compact" (Item.bareItem bool))
                  <*> (snd =. Dict.member' "notes" (Item.bareItem bool))
            )
        groups <- hp5 =. HTTP.fieldDict "groups" (Dict.list "featured" text)
        sid <- hp6 =. Req.cookie' "sid" uuid
        lang <- hp7 =. Req.cookie "lang" text
        pure (apiKey, buildNum, traceId, prefs, groups, sid, lang)
    & Req.body HTTP.noContent

-------------------------------------------------------------------------------
-- Endpoint 1's responses
-------------------------------------------------------------------------------

cacheTags = SList.items (Item.bareItem text)

relatedIds =
    (,) <$> (fst =. SList.item (Item.bareItem integer))
        <*> (snd =. SList.item (Item.bareItem integer))

scoreBuckets =
    (,) <$> (fst =. SList.innerList integer)
        <*> (snd =. SList.innerList integer)

batchGroups =
```

```
    (,) <$> (fst =. SList.innerListOf
                (SList.innerItem (Item.bareItem text))
                (Params.param "lvl" integer))
        <*> (snd =. SList.innerListOf
                (SList.innerItems (Item.bareItem integer))
                (Params.param' "lvl" integer))

-- Same `&`/`BlockArguments` treatment on the response side. Same `HTTP.`
-- names as the request side above -- literally the same functions --
-- only `setCookie` (response-only) stays `Res.`.
reviewOk = Res.res200
    & Res.headers do
        let rp1 _ = ()
            rp2 (a,_,_,_,_,_,_) = a
            rp3 (_,b,_,_,_,_,_) = b
            rp4 (_,_,c,_,_,_,_) = c
            rp5 (_,_,_,d,_,_,_) = d
            rp6 (_,_,_,_,e,_,_) = e
            rp7 (_,_,_,_,_,f,_) = f
            rp8 (_,_,_,_,_,_,g) = g
        rp1 =. HTTP.contentType HTTP.JSON
        limit <- rp2 =. HTTP.field "x-ratelimit-limit" int
        rateItem <- rp3 =. HTTP.fieldItem "x-ratelimit"
            ( (,) <$> (fst =. Item.bareItem integer)
                  <*> (snd =. Item.params
                        ( (,) <$> (fst =. Params.param "window" integer)
                              <*> (snd =. Params.flag' "burst")
                        ))
            )
        tags <- rp4 =. HTTP.fieldList "cache-tags" cacheTags
        related <- rp5 =. HTTP.fieldList "x-related-ids" relatedIds
        buckets <- rp6 =. HTTP.fieldList "x-score-buckets" scoreBuckets
        groups <- rp7 =. HTTP.fieldList "x-batch-groups" batchGroups
        -- `sessionAttrs` inlined too -- no `where` to convert here, so
        -- straight BlockArguments.
        session <- rp8 =. Res.setCookie "session" uuid do
            let ap1 (a,_,_,_,_,_) = a
                ap2 (_,b,_,_,_,_) = b
                ap3 (_,_,c,_,_,_) = c
                ap4 _ = ()
                ap5 _ = ()
                ap6 (_,_,_,d,_,_) = d
                ap7 (_,_,_,_,e,_) = e
                ap8 _ = ()
                ap9 (_,_,_,_,_,f) = f
            age <- ap1 =. Attr.maxAge
            dom <- ap2 =. Attr.domain
            pth <- ap3 =. Attr.path
            ap4 =. Res.secure
            ap5 =. Res.httpOnly
            sameSite <- ap6 =. Res.attr "SameSite" (leaf :: Leaf Attr.Attributes ByteString)
            priority <- ap7 =. Res.attr' "Priority" (leaf :: Leaf Attr.Attributes ByteString)
            ap8 =. Attr.flag "Partitioned"
```

```
            debug <- ap9 =. Attr.flag' "Debug"
            pure (age, dom, pth, sameSite, priority, debug)
        pure (limit, rateItem, tags, related, buckets, groups, session)
    & Res.body (HTTP.json @Review)

getReviewContract = getReview :-> reviewOk

-- ------------------------------------------------------------------------------
-- Endpoint 2: GET /products/{productId}/reviews/tags/{tag}+  -- `segs`
-- ------------------------------------------------------------------------------

listReviews = Req.req
    & Req.method Method.GET
    & Req.path do
        Req.lit "products"
        pid <- fst =. Req.seg "productId" uuid
        Req.lit "reviews"
        Req.lit "tags"
        tags <- snd =. Req.segs text
        pure (pid, tags)
    & Req.body HTTP.noContent

listOk = Res.res200 & Res.body (HTTP.json @[Review])

listReviewsContract = listReviews :-> listOk

-- ------------------------------------------------------------------------------
-- Endpoint 3: a `Cases` contract with two response alternatives -- `Cases`,
-- `cases`, `getResponses`, `parseResponses`, `printResponses`
-- ------------------------------------------------------------------------------

-- | Two equally valid fixes for the `path`/`URI` collision: qualify the
--   field name in record-update syntax (`Req.reqDELETE { Req.path = ... }`,
--   verified to compile fine), or `&`-pipe into the narrowing function
--   directly, shown here.
deleteReviewReq = Req.reqDELETE
    & Req.path do
        Req.lit "products"
        pid <- fst =. Req.seg "productId" uuid
        Req.lit "reviews"
        rid <- snd =. Req.seg "reviewId" int
        pure (pid, rid)

deletedRes = Res.res204 & Res.body HTTP.noContent
notFoundRes = Res.res404 & Res.body (HTTP.json @Text)

data DeleteReviewResponses f
    = Deleted (f Res.S204 Types.ResponseHeaders (IO HTTP.None))
    | ReviewNotFound (f Res.S404 Types.ResponseHeaders (IO Text))
    deriving (Generic, Cases)

deleteReviewCases = cases @DeleteReviewResponses deletedRes notFoundRes
```

```haskell
deleteReviewContract = deleteReviewReq :-< deleteReviewCases

-- | How many branches this `Cases` value actually has right now.
deleteReviewBranchCount :: Int
deleteReviewBranchCount = length (getResponses deleteReviewCases)

-- | Render the `Deleted` branch straight to a real 'Wai.Response' -- and
--   parse an incoming one back into whichever branch it actually matches.
--   Neither `Okapi.Record.Data` nor `Network.Wai` is reachable via `Okapi`
--   at all.
demoPrintDeleted :: IO Wai.Response
demoPrintDeleted = printResponses deleteReviewCases (Deleted (Data.Response Res.S204 [] (pure HTTP.None)

demoParseIncoming waiResponse = parseResponses deleteReviewCases waiResponse
```

What this example actually demonstrates

Every narrowing function (`Req.path`/`Req.query`/`Req.headers`/`Req.body`/ `Req.method`, `Res.headers`/`Res.body`) is applied with `&` — `req & Req.path do ... & Req.query do ...` — and every `do` block that's a narrowing function's argument is inlined directly via `BlockArguments` (no separate named binding purely to hand it to the function, no parens). Where a block needed its `=.`-projection helpers (`qp1`/`hp1`/ `rp1`/etc.), those became a leading `let` instead of a trailing `where`, since `where` can't attach to a bare argument expression. `text`/`integer`/ `bool` also lost the explicit `:: Leaf BareItem.BareItem _` annotations they had in an earlier draft — verified directly that inference already flows through `Item.bareItem`/`SList.item`/`Params.param` without them, even through the deepest nesting (`batchGroups`'s `innerListOf`/ `innerItem`/`innerItems` stack).

- **Endpoint 1** (`getReviewContract`) is the dense one: a path mixing a literal-integer version segment (`seg_`), literal text segments (`lit`), and typed segments (`seg`); a query using every combinator (`param`/`param'`/`param_`/`flag'`/`list'`); request headers spanning a plain fixed-value assertion (`field_`), a `Structured.BareItem` token field, a composed `Item` (fixed marker token *plus* a real parameter, demonstrating why `bareItem_` needs a value-producing sibling to stay useful), a `Dictionary` with both required and optional members, a second `Dictionary` whose member is itself an inner list, and both request cookies. The field-family combinators are all `HTTP.`; only `cookie`/`cookie'` are `Req.` (Part 2). The response side reuses the exact same `HTTP.` names — same underlying functions — builds a parameterized `Item` header from scratch, four different `List`-shaped headers (whole-list, chained single items, chained inner lists, and the full `innerListOf`/`innerItem`/`innerItems` heterogeneous-composition RFC 9651 §3.1.1 shape), and a `Set-Cookie` combining four pre-built `Attributes` combinators, two curated ones via `Res.attr`/`Res.attr'`, and two more via qualified `Attr.flag`/`Attr.flag'`.
- **Endpoint 2** (`listReviewsContract`) exists mainly for `segs` (a trailing non-empty catch-all of tag segments) and to show `Req.method`/`Method.GET` used directly instead of one of the pre-built `reqGET` starting points — note `Req.method` (the narrowing function) takes the bare singleton `Method.GET` itself, not `Method.method  Method.GET` (which builds a full `Method` codec) — a real distinction worth not conflating, caught by the compiler when it was gotten wrong while writing this.
- **Endpoint 3** (`deleteReviewContract`) is the `Cases` demo: two response alternatives, `Req.reqDELETE & Req.path do ...` for the `path`/`URI` collision fix (3.3's other valid fix — qualifying the field name in record-update syntax instead — is documented there rather than shown twice), plus `getResponses`, and both directions of `printResponses`/`parseResponses` against a real `Wai.Response`.

Across the whole thing, thirteen qualifiers are load-bearing: `HTTP`, `Req`, `Res`, `Method`, `Query`, `Attr`, `BareItem`, `Item`, `SList`, `Dict`, `Params`, `Wai`, `Data` — plus `Struct`, imported but never actually typed in the body, since `fieldDict`/`fieldItem`/`fieldList` cover its job (Part 5). That's one more than the previous iteration of this document (`HTTP` is new), but it's a more honest split: `field`/`json`/`contentType` now mean exactly one thing regardless of which qualifier reaches them, because there's only one qualifier that reaches them. A

module that only builds *one* side still pays a two-import minimum (its facade plus `HTTP`) — no configuration gets back to a single unqualified import for the HTTP DSL any more, and that's the deliberate tradeoff of this design (Part 3).